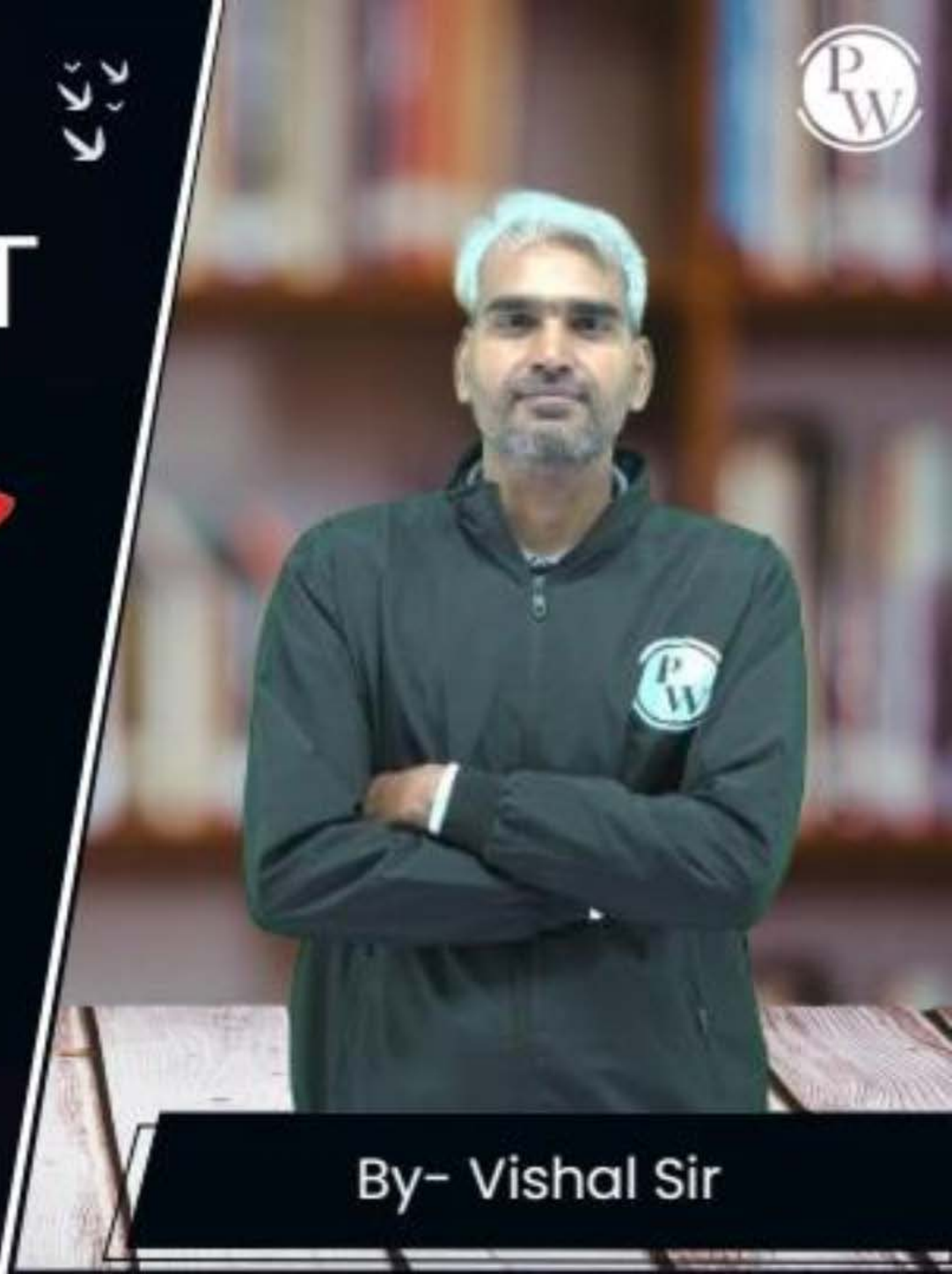
Computer Science & IT



Database Management System

Transaction & concurrency control

Lecture No. 03



By- Vishal Sir

Recap of Previous Lecture



- Topic: Durability
- Topic: Isolation
- Topic: Schedule (Serial schedule & Concurrent schedule)
- Topic: Serializable schedule

Slide

Topics to be Covered



- Topic: Consistency
- Topic: Equivalent schedule
- Topic: Serializable schedule
- Topic: RW problem
- Topic: WR problem

Slide



Topic : Transaction

A sequence of logically related operations that are used to perform a particular task is called a transaction

Read (A) :- This operation is used to read the value of dataitem 'A' from database (i.e. disk) to main memory

Write (A) :- The current value of dataitem A that is present in main memory will be updated into the database

* Let 'A' and 'B' represent the amount in two different accounts. A & B respectively
  - Consider, initially A = 1000, & B = 0
  - Transaction to transfer 500/- from A to B

Transaction

Read (A): Because of this oph a copy of dataitem A will be created in Main memory

A = A - 500; This oph will be performed only in main memory

Write (A) Current value of dataitem A from the M.M. will be updated into database

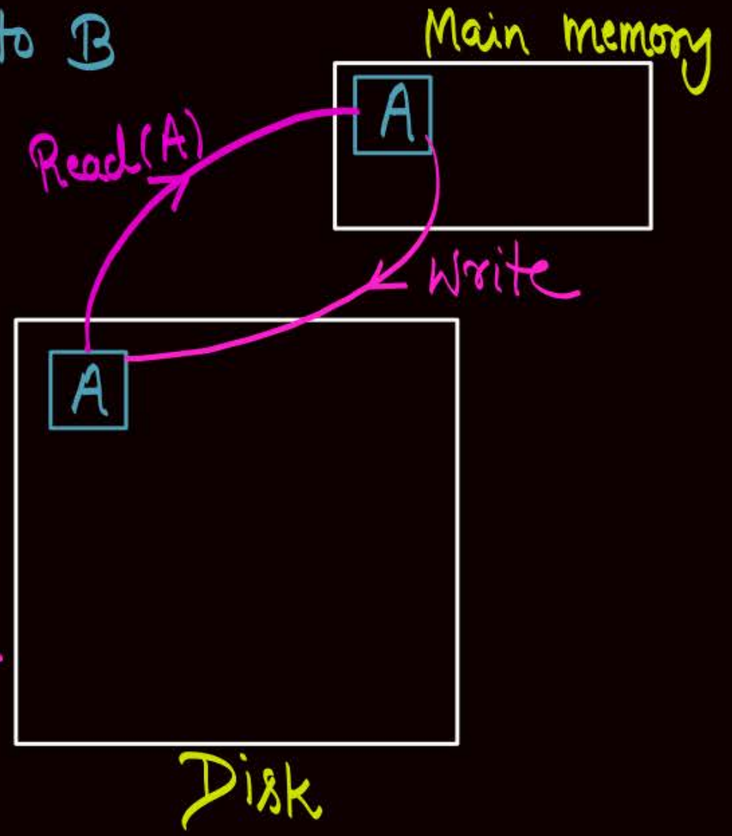
Read (B)

B = B + 500

Write (B)

Commit { Transaction Committed means transaction executed successfully, from user point of view }

These two oph are not important from database Point of view





Topic : Blind write

{ When a transaction T updates a dataitem without reading that dataitem from database }

R(A) = Read (A)
W(A) = Write (A)

Transaction (T)

R(A)
R(C)
W(A) ← It is not a blind write oph because 'T' has read the dataitem 'A' from the database before this W(A) oph

R(D)
W(B) ← It is blind write oph because transaction 'T' did not read the value of dataitem 'B' before this W(B) oph

Not a blind write → W(D)

Consider two transactions T_1 & T_2

T_1
R(A)
R(B)

T_2
R(B)
R(A)

T_1 & T_2 are two different transactions

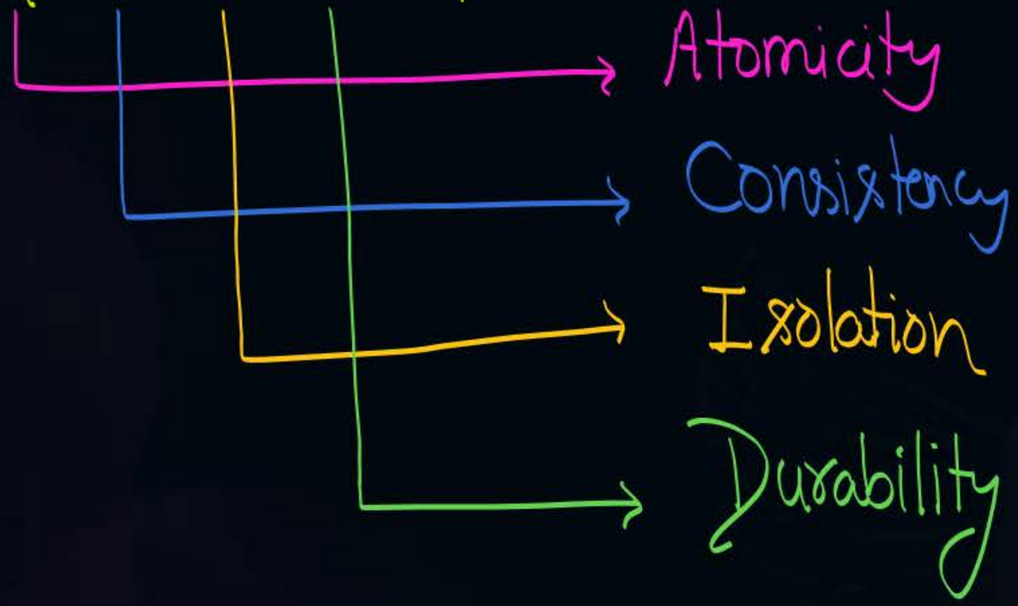
⇓

We can not change the position of the operations in the same transaction.



Topic : ACID properties

* For the integrity of the database A C I D properties must be satisfied





Topic : Atomicity

* Atomicity states that, either execute all operations of a transaction or none of them.

There are two problems

1. We can not execute all the operations of a transaction at once, they will be executed in a sequence one after another
2. If we start executing the operations of a transaction, then we can not guarantee that all operations of that transaction will get executed without any failure

* Let 'A' and 'B' represent the amount in two different accounts. A & B respectively
  - Consider, initially A = 1000, & B = 0
  - Transaction to transfer 500/- from A to B

Transaction

Read (A)

A = A - 500;

Write (A)

Read (B)

B = B + 500

Write (B)

Commit

These two opⁿ are not important from database Point of view

* failure occur in between

* If we start executing the operations of this transaction, and if failure occur before executing the Commit operation, then atomicity is violated

{ Transaction Committed means transaction executed successfully, from user point of view }

Note :-

To ensure atomicity, if transaction failed before executing its commit operation successfully, then we must undo all the changes performed by the transaction before its failure

{ If we are able to do so, then it will be similar to the situation when none of the operation of the transaction is executed }

* The process of undoing the changes performed by a transaction is called rollbacking of that transaction

To ensure atomicity, "Recovery management Component" is used



Topic : Recovery management component

* Recovery management Component is responsible for "undo" and "redo" operation

"undo" oph will be performed w.r.t. failed transaction

"redo" oph will be performed w.r.t. Committed transaction

↳ The process of undoing the changes performed by a failed transaction is rollback

Note:- For the purpose of "undo" and "redo" operation transaction log is maintained by recovery management Component



Topic : Transaction log

* Transaction log is used to record the activities performed by the transactions

* Let 'A' and 'B' represent the amount in two different accounts A & B respectively
  - Consider, initially A = 1000, & B = 0
  - Transaction to transfer 500/- from A to B

Transaction (T_1)

Read (A)

A = A - 500;

Write (A)

Read (B)

B = B + 500

Write (B)

Commit

These two opⁿ are not important from database Point of view

Transaction Log

Start : T_1

(T_1, A, 1000, 500)

If transaction T_1 fails then we need to rollback the value of dataitem A to '1000'

Consider the following representation

(T_i, X, V_1, V_2)

T_i → Transaction number

X → dataitem

V_1 → old value of dataitem

V_2 → new value of dataitem



Topic : Transaction log

* Transaction log is used to record the activities performed by the transactions
  * We can recover from a failure only if "transaction log" is available
    ↳ ∴ Transaction log is maintained by recovery management Component in the secondary memory (disk).

Note:
Either we can maintain
a seperate transaction log
for each ongoing transaction
(or) we can maintain a
unified transaction log

Consider: (T_i, X, V_1, V_2)

- T_i → Transaction no.
- X → data item
- V_1 → old value
- V_2 → new value

and, Consider the following "transaction log"

T_1 : Start
(T_1, A, 200, 120)
(T_1, B, 100, 150)
T_2 : Start
(T_2, C, 250, 75)
Check point → The operations Performed before the check point have been updated into database
(T_1, D, 90, 140)
(T_2, E, 500, 290)
T_2 : Commit
T_3 : Start
(T_3, F, 280, 100)

Let System Fails at this point! →

→ T_1 & T_3 were the ongoing transactions when failure occured {i.e. they failed before Commit}
↳ we need to undo all the opn Performed by T_1 & T_3

If there is any transaction that Committed before the last check point then we don't need to worry about those transaction

If any transaction/s Committed after the last check point, then we need to "redo" only those operations of that transaction that appeards after last check point



Topic : Durability

Changes performed by committed transactions must persist in the database even if the failure occur

Read about
RAID architecture



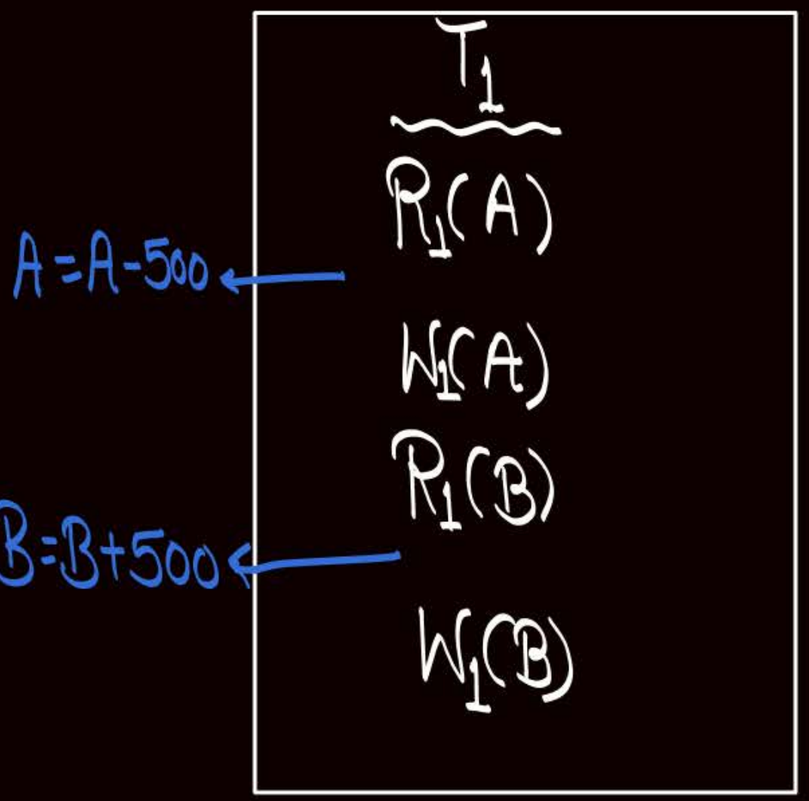
Topic : Isolation

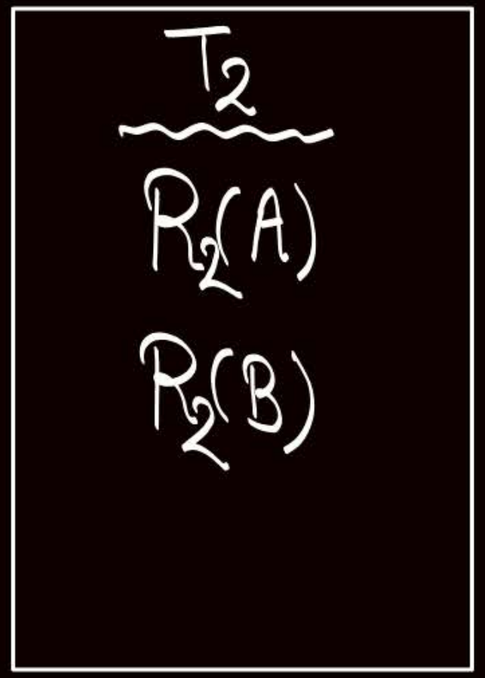
* Isolation says that if two or more transactions are executing Concurrently, then they all must be unaware of each other.

eg:→ Consider two transactions.

T_1: Transfer Rs 500/- from account A to B

T_2: Check the balance amount in account A & B respectively





$R_i(X)$: Read of dataitem X by transaction T_i

$W_j(y)$: Write of dataitem y by transaction T_j



Topic : Schedule

Time ordered sequence of operations of two or more transaction is called a schedule

eg.

Schedule 'S'

T_1	T_2
$R_1(A)$	
	$R_2(A)$
$W_1(A)$	
	$R_2(B)$
$R_1(B)$	
$W_1(B)$	

time increasing

$S \equiv R_1(A); R_2(A); W_1(A); R_2(B); R_1(B); W_1(B)$

* Consider the following Schedule

S: $W_2(B), R_1(A), R_2(A), W_3(A), R_2(B), W_1(B), R_3(B)$

≡

S

T_1	T_2	T_3
	$W_2(B)$	
$R_1(A)$		
	$R_2(A)$	
		$W_3(A)$
	$R_2(B)$	
$W_1(B)$		
		$R_3(B)$



Topic : Schedule

There are two types of schedules

1. Serial schedule (one after another)
2. Concurrent schedule (interleaved execution of opn is allowed)



Topic : Serial Schedule

(One after another)

We can start executing the operations of a new transaction only if we have executed all the operations of previously started transaction

Such schedules are called serial schedules

- Consider two transactions T_1 & T_2

  T_1: $R_1(A)$, $W_1(A)$, $R_1(B)$, $W_1(B)$ {transfer Rs 500/- from A to B}

  T_2: $R_2(A)$, $R_2(B)$ {check balance amount in A & B respectively}

- If there are two transactions, then only two serial schedules are possible {(i) $T_1 \rightarrow T_2$: First Execute T_1, then T_2, (ii) $T_2 \rightarrow T_1$: First Execute T_2, then T_1

$S_1 \equiv T_1 \rightarrow T_2$

T_1	T_2
$R_1(A)$	
$W_1(A)$	
$R_1(B)$	
$W_1(B)$	
	$R_2(A)$
	$R_2(B)$

$S_2 \equiv T_2 \rightarrow T_1$

T_1	T_2
	$R_2(A)$
	$R_2(B)$
$R_1(A)$	
$W_1(A)$	
$R_1(B)$	
$W_1(B)$	

Note: If there are n transactions, then $n!$ serial schedules are possible

Note: All serial schedules will always satisfy the isolation condition, but throughput of the system will be low with serial schedules

↳ ∴ We think about Concurrent execution



Topic : Concurrent Schedule

- If operations of two or more transactions are allowed to get executed in interleaved manner, then such schedule are called Concurrent schedules

- Consider two transactions T_1 & T_2

  T_1: $R_1(A)$, $W_1(A)$, $R_1(B)$, $W_1(B)$ { transfer Rs 500/- from A to B }

  T_2: $R_2(A)$, $R_2(B)$ { Check balance amount in A & B respectively }

S_1

T_1	T_2
$R_1(A)$	
$W_1(A)$	
	$R_2(A)$
	$R_2(B)$
$R_1(B)$	
$W_1(B)$	

S_2

T_1	T_2
$R_1(A)$	
$W_1(A)$	
	$R_2(A)$
$R_1(B)$	
$W_1(B)$	
	$R_2(B)$

- Consider two transactions T_1 & T_2

Let, initially A = 1000 & B = 0

T_1: $R_1(A)$, $W_1(A)$, $R_1(B)$, $W_1(B)$ { transfer Rs 500/- from A to B }

T_2: $R_2(A)$, $R_2(B)$ { check balance amount in A & B respectively }

output of Every Serial Schedule is Considered Correct

$S_1 \equiv T_1 \rightarrow T_2$

T_1	T_2
$R_1(A)$ ← A = 1000	
$W_1(A)$ ← A = 500	
$R_1(B)$ ← B = 0	
$W_1(B)$ ← B = 500	
	$R_2(A)$ → 500
	$R_2(B)$ → 500

$S_2 \equiv T_2 \rightarrow T_1$

T_1	T_2
	$R_2(A)$ → A = 1000
	$R_2(B)$ → B = 0
$R_1(A)$ ← A = 1000	
$W_1(A)$ ← A = 500	
$R_1(B)$ ← B = 0	
$W_1(B)$ ← B = 500	

- Consider two transactions T_1 & T_2

T_1: $R_1(A)$, $W_1(A)$, $R_1(B)$, $W_1(B)$ {transfer Rs 500/- from A to B}

T_2: $R_2(A)$, $R_2(B)$ {check balance amount in A & B respectively}

Note:- Throughput of the system will increase with Concurrent schedule but, Every Concurrent schedule need not satisfy the isolation Cond^n.

(S1)

T_1	T_2	
$R_1(A)$		A=1000
$W_1(A)$		A=500
	$R_2(A)$	A=500
	$R_2(B)$	B=0
$R_1(B)$		B=0
$W_1(B)$		B=500

This values are neither same as serial schedule $T_1 \to T_2$ ▲ nor, same as serial schedule $T_2 \to T_1$ ▲

⇓

In this case T_2 will be able to identify that it is executing along with some other transaction ⇒ Hence, isolation Cond^n is not satisfied by schedule (S1)

(S2)

T_1	T_2	
$R_1(A)$		A=1000
$W_1(A)$		A=500
	$R_2(A)$	A=500
$R_1(B)$		B=0
$W_1(B)$		B=500
	$R_2(B)$	B=500

All values are exactly same as serial schedule $T_1 \to T_2$

⇓

∴ None of the transaction will be able to identify that they are executing along with some other transactions

⇓

Hence, isolation Cond^n is satisfied by schedule (S2)

Note :-

In order to say that schedule "S" satisfies the isolation Condition, behaviour of schedule "S" must be equivalent to at least one of the Serial schedule.



Topic : Isolation

* Isolation says that if two or more transactions are executing concurrently, then they all must be unaware of each other.

(or)

* In order to say that a schedule 'S' satisfies isolation condition, behaviour of schedule S must be equivalent to at least one serial schedule.

* Serializable Schedule :→

A schedule is called a serializable schedule if and only if, behaviour of schedule is equivalent to at least one of the "serial schedule"

* Note: ⇒ Every serial schedule is serializable schedule, but Every serializable schedule need not be a serial schedule



Topic : Isolation

* Isolation says that if two or more transactions are executing concurrently, then they all must be unaware of each other.

(or)

* In order to say that a schedule 'S' satisfies isolation condition, behaviour of schedule S must be equivalent to at least one serial schedule.

(or)

* In order to say that schedule 'S' satisfies isolation condn, schedule 'S' must be a serializable schedule

- Consider two transactions T_1 & T_2

T_1: $R_1(A)$, $W_1(A)$, $R_1(B)$, $W_1(B)$ {transfer Rs 500/- from A to B}

T_2: $R_2(A)$, $R_2(B)$ {check balance amount in A & B respectively}

(S1)

T_1	T_2	
$R_1(A)$		A=1000
$W_1(A)$		A=500
	$R_2(A)$	A=500
	$R_2(B)$	B=0
$R_1(B)$		B=0
$W_1(B)$		B=500

It is not same as $T_1 \rightarrow T_2$

∴ S_1 is not Equivalent to $T_1 \rightarrow T_2$

i.e, $S_1 \not\equiv T_1 \rightarrow T_2$

It is not same as $T_2 \rightarrow T_1$,

∴ S_1 is not Equivalent to $T_2 \rightarrow T_1$

i.e, $S_1 \not\equiv T_2 \rightarrow T_1$

"S_1" is not equivalent to any of the serial schedule

Hence, "S_1" is a non-serializable schedule

(S2)

T_1	T_2	
$R_1(A)$		A=1000
$W_1(A)$		A=500
	$R_2(A)$	A=500
$R_1(B)$		B=0
$W_1(B)$		B=500
	$R_2(B)$	B=500

It is not same as serial schedule $T_2 \rightarrow T_1$

∴ S_2 is not Equivalent to $T_2 \rightarrow T_1$

i.e, $S_2 \not\equiv T_2 \rightarrow T_1$

All these values are same as serial schedule $T_1 \rightarrow T_2$

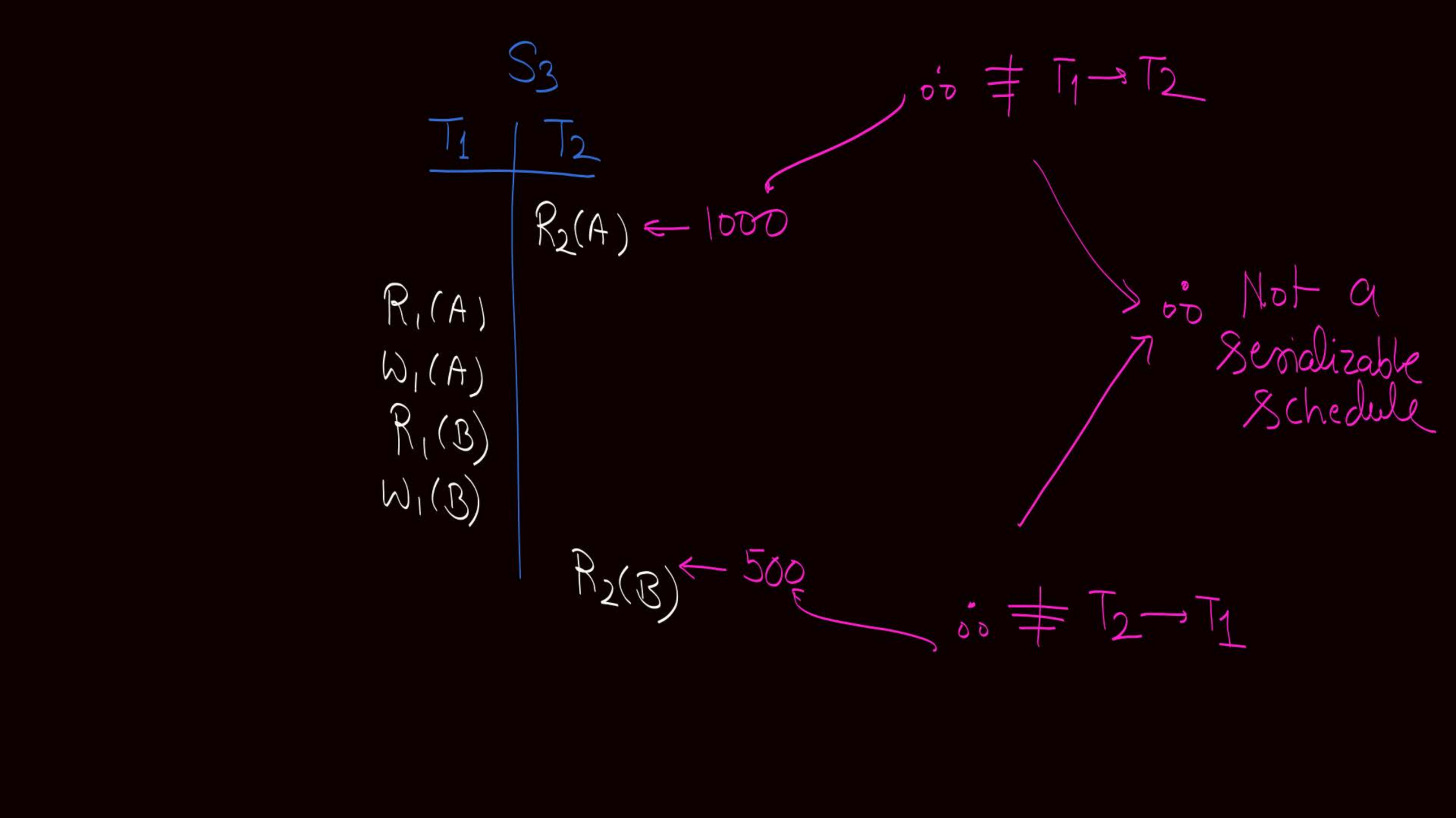
∴ S_2 is Equivalent to serial schedule $T_1 \rightarrow T_2$

i.e, $S_2 \equiv T_1 \rightarrow T_2$

$S_2 \not\equiv T_2 \rightarrow T_1$, but $S_2 \equiv T_1 \rightarrow T_2$

∴ S_2 is equivalent to at least one serial schedule

Hence, S_2 is a serializable schedule


S3
T1
T2
R2(A) ← 1000
R1(A)
W1(A)
R1(B)
W1(B)
R2(B) ← 500
∴ ≠ T1 → T2
∴ ≠ T2 → T1
∴ Not a Serializable Schedule

Note : Concurrency Control Component is responsible for avoiding the execution of non-serializable schedule

i.e, Concurrency Control Component ensures serializability

i.e, Concurrency Control Component ensures isolation



Topic : Consistency

Consistency states that, before and after the execution of transactions of a schedule, the database must remain Consistent

Criteria for Consistency :-

1. Schedule must be strict recoverable.

and

2. Schedule must be Serializable Schedule

Note:

1. For atomicity and durability, Recovery management Component comes into the picture
2. For isolation, Concurrency Control component is responsible
3. For Consistency, both recovery management Component and Concurrency Control Component are responsible

Q:- Consider two transactions.

T_1: $R_1(A)$, $W_1(A)$, $R_1(B)$, $W_1(B)$

&

T_2: $R_2(A)$, $R_2(B)$

How many Concurrent schedules are possible over transactions T_1 & T_2

$$^{n}C_{\gamma} = {}^{n}C_{n-\gamma}$$

T_1	T_2

No. of Concurrent schedules

$$= \frac{(4+2)!}{4!\,2!} = 15$$

$$= {}^{6}C_{4} * 1 * {}^{2}C_{2} * 1$$

$$= 15 * 1 = 15$$

(or)

$$= {}^{6}C_{2} * 1 * {}^{4}C_{4} * 1$$

$$= 15 * 1 = 15$$

Q:- Let T_1, T_2, & T_3 are three transactions with m, n and p operations respectively.
How many Concurrent schedules are possible over three transactions T_1, T_2 & T_3

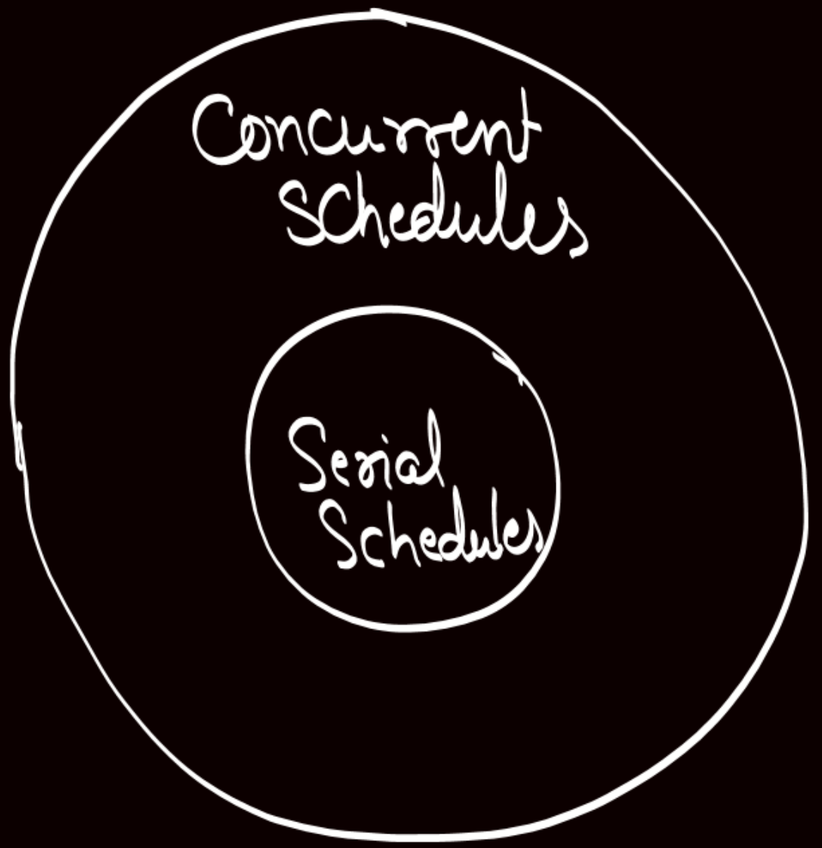
Soln.

$$\text{No. of Concurrent schedules} = {}^{(m+n+p)}C_m * 1 * {}^{(n+p)}C_n * 1 * {}^{p}C_p * 1 = \frac{(m+n+p)!}{m! \cdot n! \cdot p!}$$

How many Concurrent non-serial schedules are possible

$$\text{Ans} = \frac{(m+n+p)!}{m!\ n!\ p!} - 3!$$

↳ Subtract no. of Serial schedules



All serial schedules are included in all Concurrent Schedules



Topic : Equivalent schedule

For two schedules to be equivalent, following two conditions must be satisfied

1. Every corresponding read operation must be same in both schedules.

and

2. Final update (final write) for every corresponding dataitem must be same in both schedules.

Q. Consider two transactions,

T_1 : $R_1(A)$, $W_1(A)$, $R_1(B)$, $W_1(B)$

T_2 : $R_2(A)$, $R_2(B)$

and following two schedules S_1 & S_2 over transactions T_1 & T_2

S_1

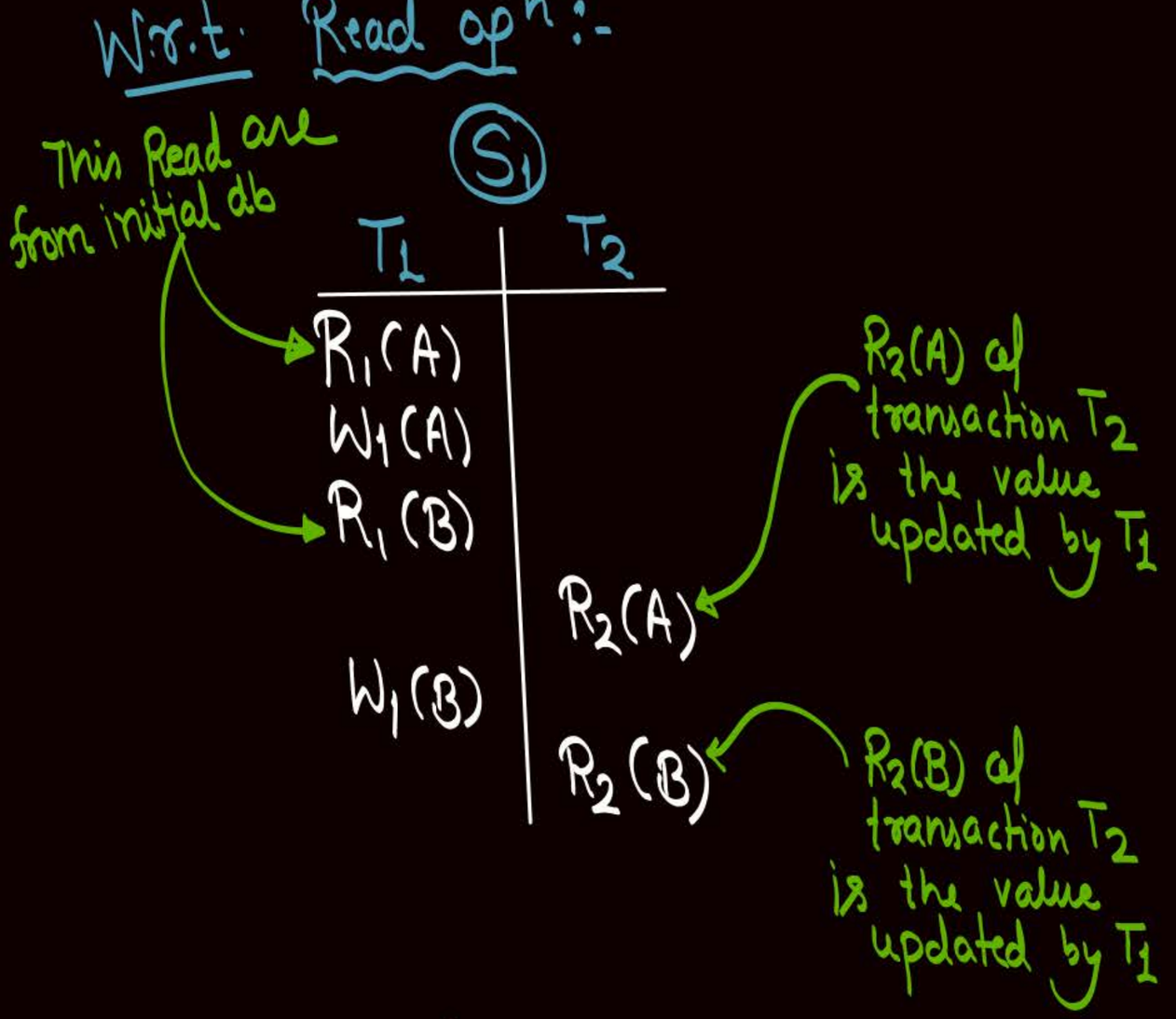
T_1	T_2
$R_1(A)$	
$W_1(A)$	
$R_1(B)$	
	$R_2(A)$
$W_1(B)$	
	$R_2(B)$

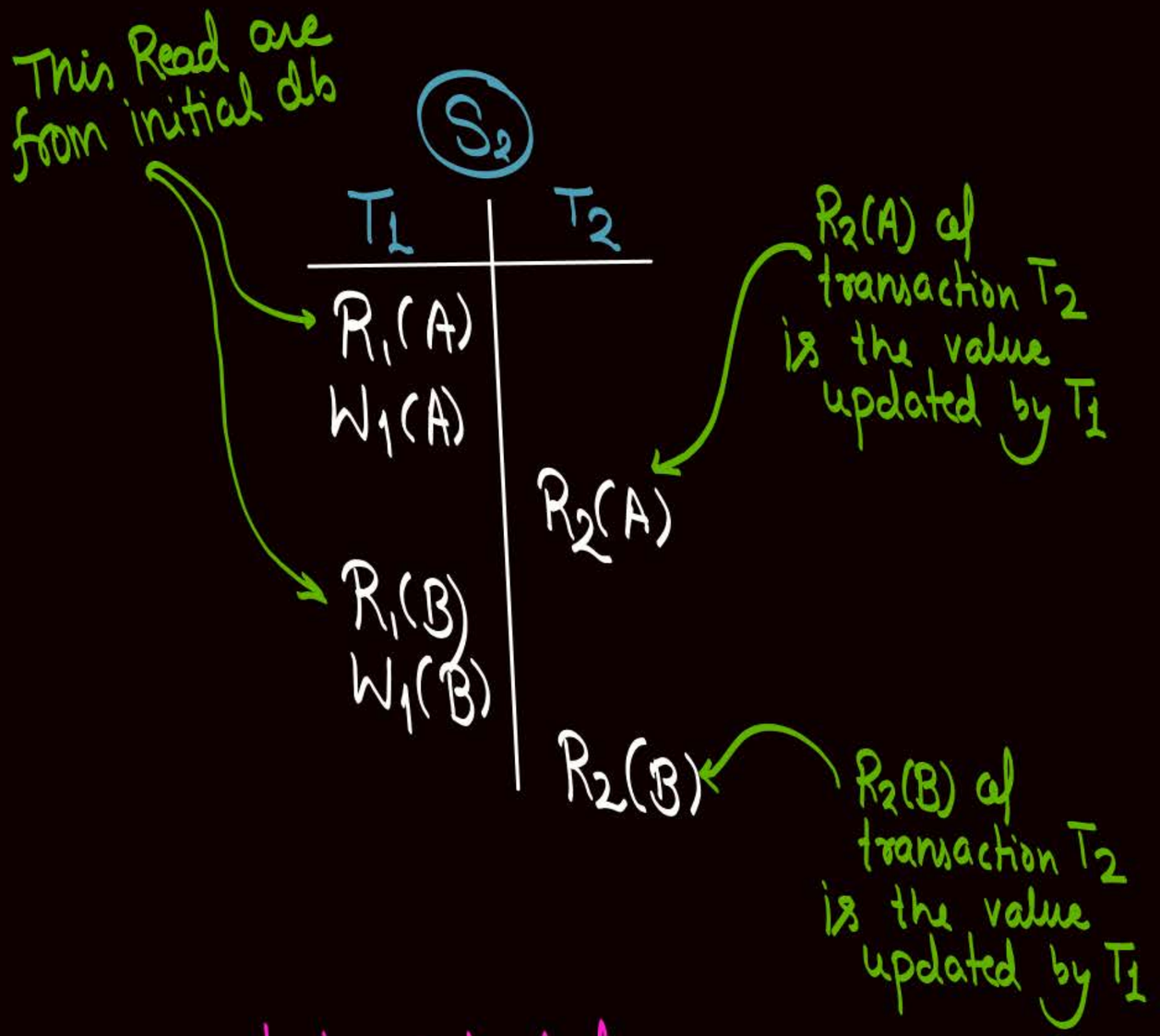
S_2

T_1	T_2
$R_1(A)$	
$W_1(A)$	
	$R_2(A)$
$R_1(B)$	
$W_1(B)$	
	$R_2(B)$

Check whether schedule S_1 & S_2 are equivalent or not.

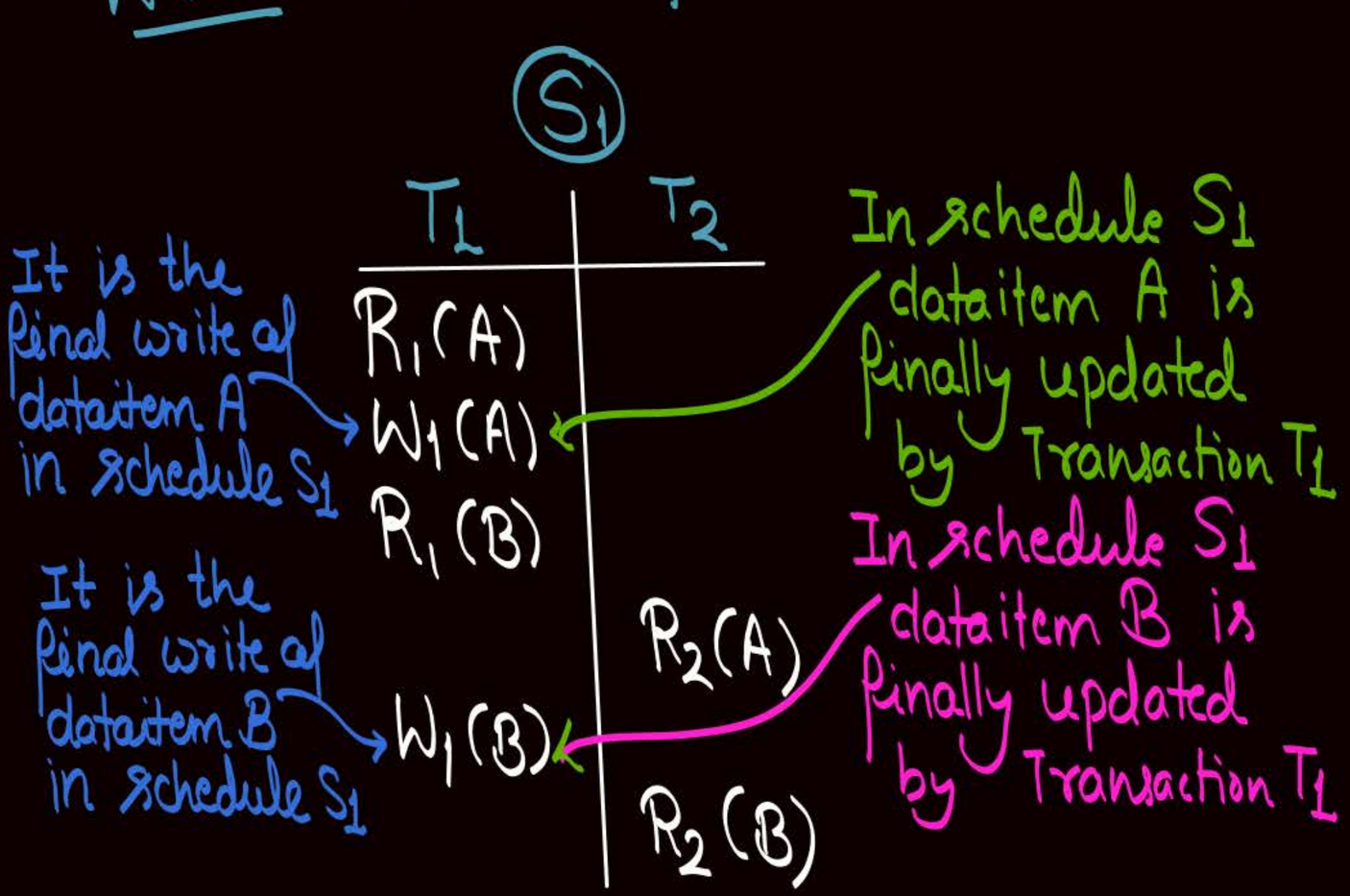
W.r.t: Read opⁿ :-

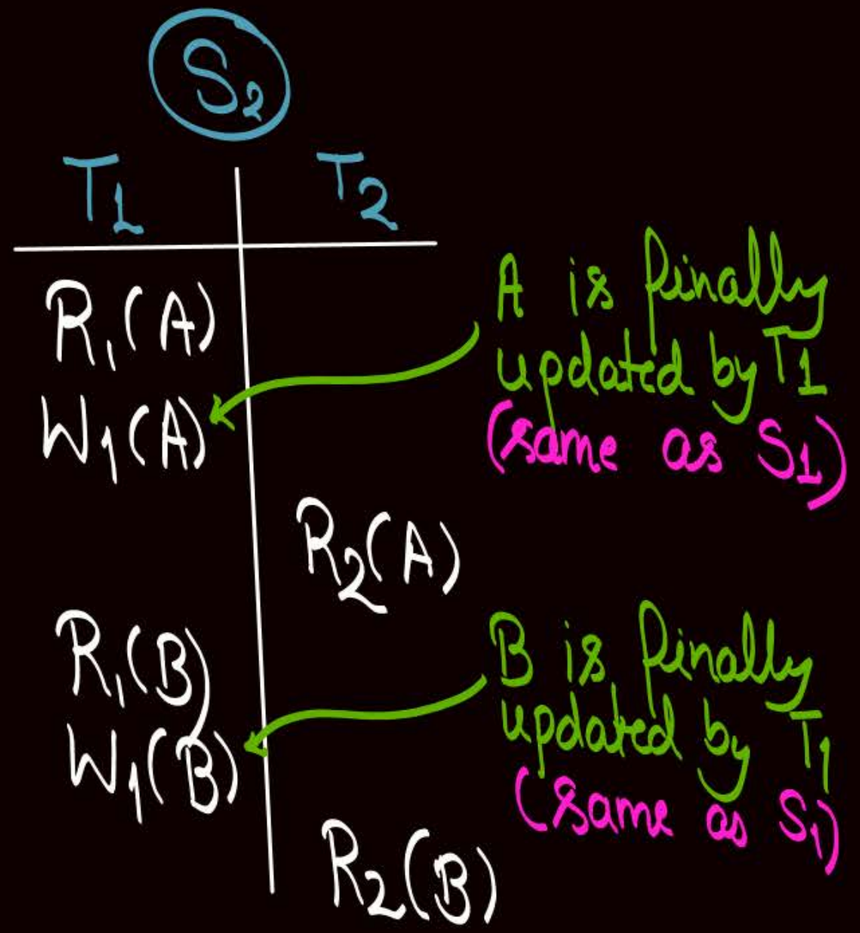




Note: all read oph are exactly same in both schedules

W.r.t: Write opn





* Final update of corresponding dataitem is also same in both the schedules.

Q. Consider two transactions,

T_1 : $R_1(A)$, $W_1(A)$, $R_1(B)$, $W_1(B)$

T_2 : $R_2(A)$, $R_2(B)$

and following two schedules S_1 & S_2 over transactions T_1 & T_2

Check Whether Schedule S_1 & S_2 are equivalent or not

S_1

T_1	T_2
$R_1(A)$	
$W_1(A)$	
$R_1(B)$	
	$R_2(A)$
$W_1(B)$	
	$R_2(B)$

S_2

T_1	T_2
$R_1(A)$	
$W_1(A)$	
	$R_2(A)$
$R_1(B)$	
$W_1(B)$	
	$R_2(B)$

We observed,

① Every Corresponding Read is identical in S_1 & S_2

& ② Every final update is also identical in S_1 & S_2

∴ S_1 & S_2 are equivalent

$S_1 \equiv S_2$

* Serializable Schedule :→

A schedule is called a serializable schedule if and only if, behaviour of schedule is equivalent to at least one of the "serial schedule"

Q:- Check whether the schedule is a serializable schedule or not ?



T_1	T_2
	$R_2(A)$
$W_1(A)$	
$W_1(B)$	
	$R_2(B)$

from initial db

Value updated by T_1

Serial schedules w.r.t. transactions of schedule S_3

$T_1 \rightarrow T_2$

T_1	T_2
$W_1(A)$	
$W_1(B)$	
	$R_2(A)$
	$R_2(B)$

Value updated by T_1 (Not same as S_3)

$\therefore S_3 \not\equiv T_1 \rightarrow T_2$

$T_2 \rightarrow T_1$

T_1	T_2
	$R_2(A)$
	$R_2(B)$
$W_1(A)$	
$W_1(B)$	

Value from initial db (Not same as S_3)

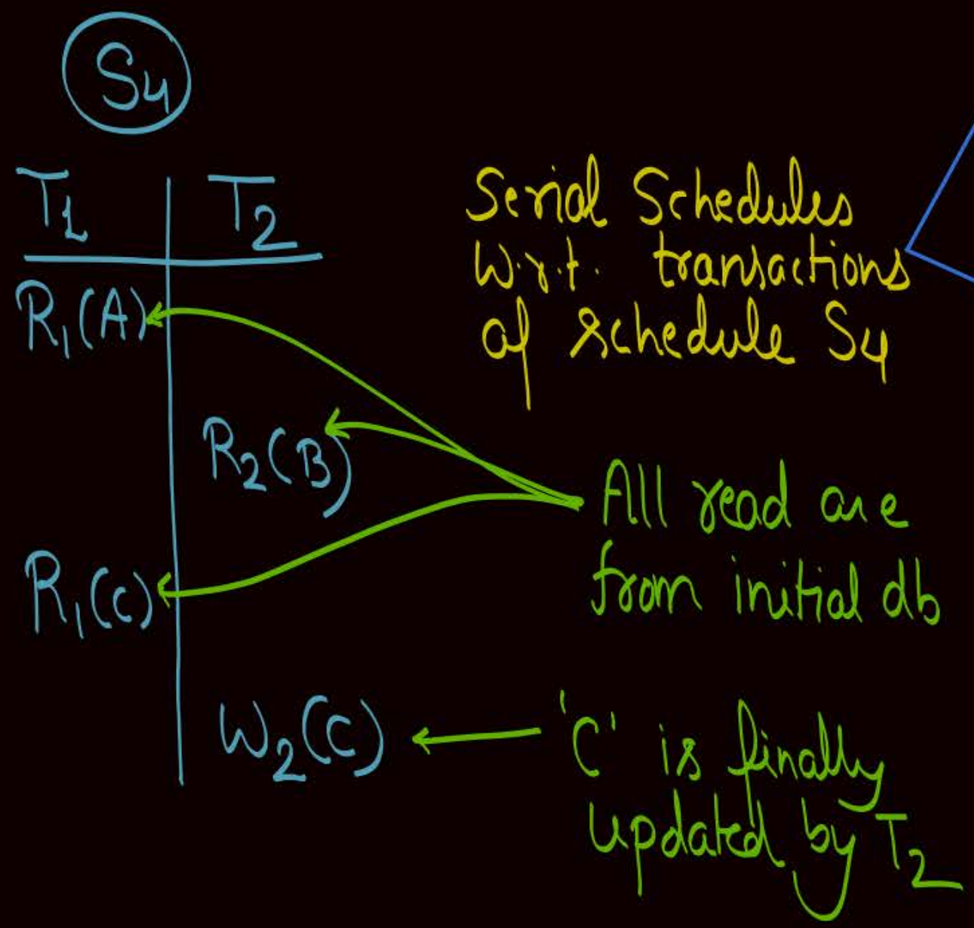
$\therefore S_3 \not\equiv T_2 \rightarrow T_1$

Schedule S_3 is not Equivalent to any of the serial schedules

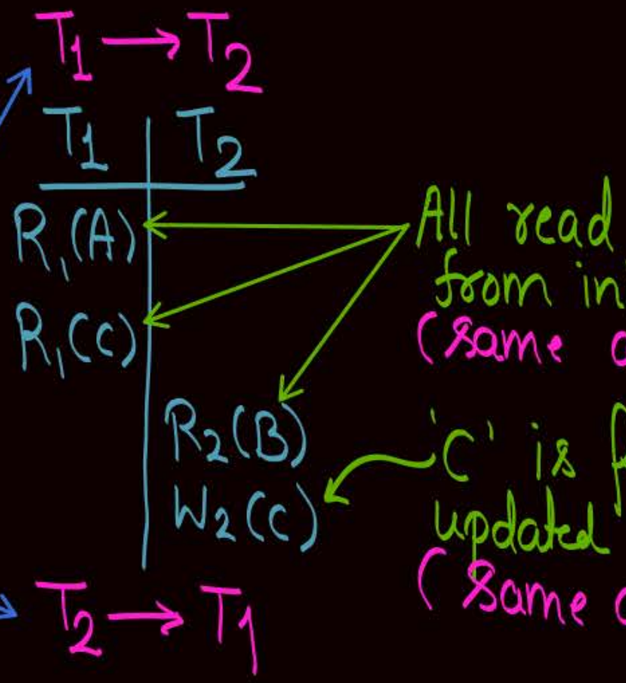
$\Downarrow$

Hence, S_3 is a non-serializable schedule

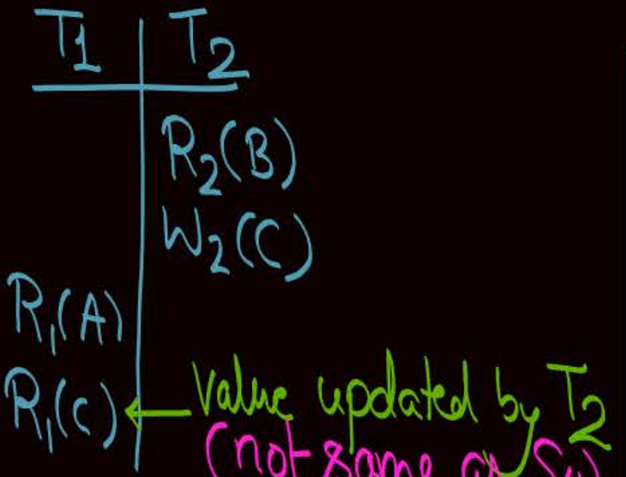
Q:- Check whether the schedule is a serializable schedule or not?



Serial Schedules w.r.t. transactions of schedule S4



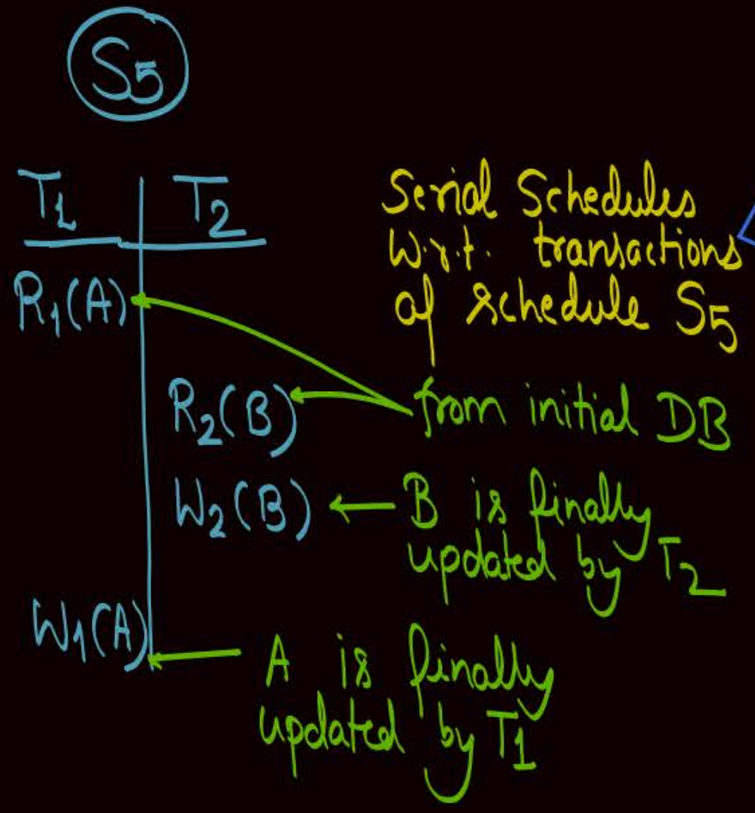
Every read is same as S4,
↕
Every final update is same as S4
∴ $S_4 \equiv T_1 \rightarrow T_2$



∴ $S_4 \not\equiv T_2 \rightarrow T_1$

$S_4 \not\equiv T_2 \rightarrow T_1$, but $S_4 \equiv T_1 \rightarrow T_2$,
'S4' is equivalent to at least one serial schedule
∴ S4 is a serializable schedule, and Equivalent serial schedule is $T_1 \rightarrow T_2$

Q:- Check whether the schedule is a serializable schedule or not?



Serial Schedules w.r.t. transactions of schedule S_5



Read from initial db (same as S_5)

A is finally updated by T_1 (same as S_5)

B is finally updated by T_2 (same as S_5)

Read from initial db (same as S_5)

B is finally updated by T_2 (same as S_5)

A is finally updated by T_1 (same as S_5)

$\therefore S_5 \equiv T_1 \rightarrow T_2$

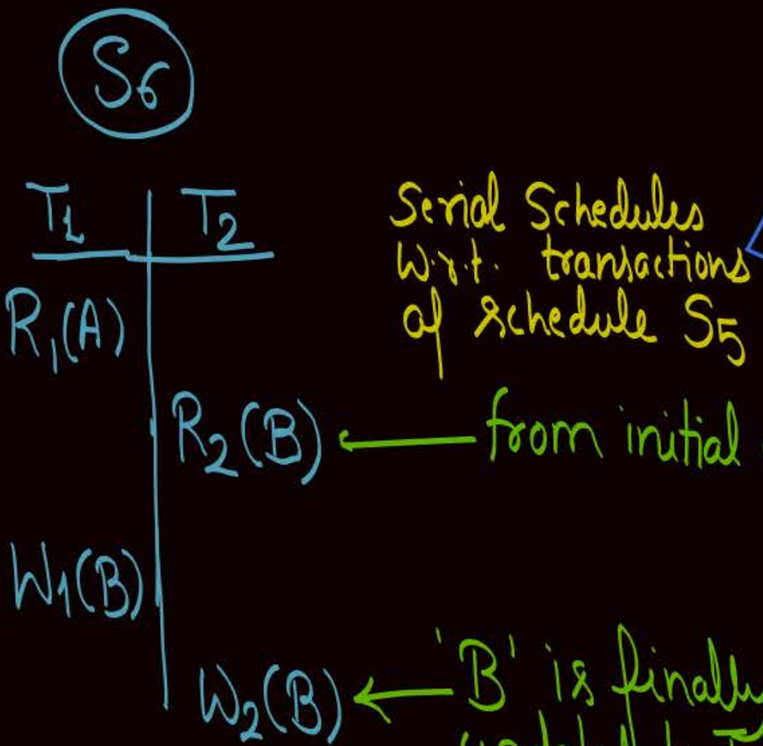
$\therefore S_5 \equiv T_2 \rightarrow T_1$

Schedule S_5 is Equivalent to at least one serial schedule

$\therefore S_5$ is a serializable schedule

There are two equivalent serial schedule $T_1 \rightarrow T_2$ & $T_2 \rightarrow T_1$

Q:- Check whether the schedule is a serializable schedule or not?



Serial Schedules w.r.t. transactions of schedule S_5

$T_1 \rightarrow T_2$

T_1	T_2
$R_1(A)$	
$W_1(B)$	
	$R_2(B)$
	$W_2(B)$

$R_2(B)$ ← Value updated by T_1 (∵ Not same as S_6)

∴ $S_6 \not\equiv T_1 \rightarrow T_2$

$T_2 \rightarrow T_1$

T_1	T_2
	$R_2(B)$
	$W_2(B)$
$R_1(A)$	
$W_1(B)$	

$R_1(A)$ ← 'B' is finally updated by T_1 (∵ Not same as S_6)

∴ $S_6 \not\equiv T_2 \rightarrow T_1$

$R_2(B)$ ← from initial db

$W_2(B)$ ← 'B' is finally updated by T_2

S_6 is not Equivalent to any of the serial schedule

⇓

Hence, S_6 is a non-serializable schedule



2 mins Summary

- Topic: Consistency
- Topic: Equivalent schedule
- Topic: Serializable schedule
- Topic: RW problem
- Topic: WR problem

Slide

THANK - YOU